FUTURE INTERNS

GET TRAINED • WORK ON LIVE PROJECTS • EARN CERTIFICATES

contact@futureinterns.com futureinterns.com

Vulnerability Assessment Report For http://demo.testfire.net

Email: hagarnofal1@gmail.com

Phone: 01020787064

Table of Contents:

1. Executive Summary.....3

2. Scope.....3

3. Methodology.....3

4. Exposed Services.....4

5. Vulnerabilities Overview.....5

6. Detailed Findings.....6

SQL Injection.....6

Cross-Site Scripting.....7

Missing Content Security.....8

Missing Anti-Clickjacking Protection.....9

Cookie Without SameSite Attribute.....10

Information Disclosure.....11

Missing Subresource Integrity.....12

Security Misconfiguration.....13

7. Conclusion.....14

Executive Summary

This report presents the results of a security assessment conducted on a public web application. Several vulnerabilities were identified, including high-risk issues such as SQL Injection and Cross-Site Scripting (XSS), which could allow attackers to access sensitive data or manipulate the application. Immediate remediation is recommended.

Scope	
Target	http://demo.testfire.net
Type	Public Web Application
Approach	Passive Scanning Only

Tools Used:

- **Nmap**
- **OWASP ZAP**
- **Browser Dev-Tools**

▢ Exposed Services (Nmap Scan)

- **Risk:** Informational
- **Description:**
  A network scan was performed using Nmap to identify exposed services and open ports on the target web server. The results show that the application is accessible through multiple open ports, which increases the attack surface.
- **Evidence:**
  Nmap scan results identified the following open ports and services:
- **80/tcp (HTTP)** – Web traffic
- **443/tcp (HTTPS)** – Secure web traffic
- **8080/tcp (HTTP Proxy)** – Alternative web service
- **Note:** "Open ports are not vulnerabilities, but they increase the attack surface."

```
┌──(kali㉿kali)-[~]
└─$ nmap demo.testfire.net
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-29 05:17 EDT
Nmap scan report for demo.testfire.net (65.61.137.117)
Host is up (0.025s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE
80/tcp   open  http
443/tcp  open  https
8080/tcp open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 54.76 seconds
```

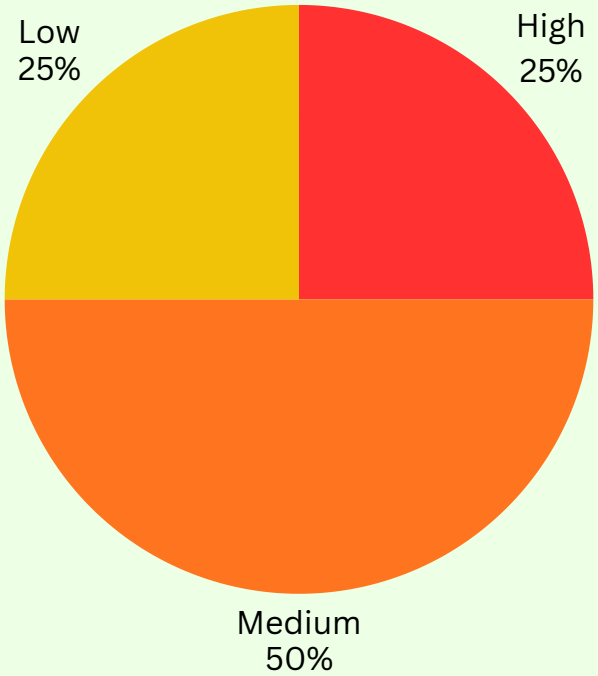
Figure 1: Nmap scan output showing open ports (80, 443, 8080) and associated services on the target server.

- **Impact:**
  While open ports are not vulnerabilities by themselves, unnecessary exposure may provide attackers with additional entry points if services are misconfigured or vulnerable.

• Vulnerabilities Overview

High	SQL Injection Cross-Site Scripting
Medium	Missing Content Security Policy Missing Anti-Clickjacking Protection Cookie Without SameSite Attribute
Low	Information Disclosure Missing Subresource Integrity

• Risk Distribution



SQL Injection

- **Risk:** High
- **Description**:
  The application fails to properly validate and sanitize user-supplied input before incorporating it into backend SQL queries. This allows an attacker to manipulate database queries by injecting malicious SQL code through input fields such as search forms or login parameters.
- **Evidence:**
  User-controlled parameter detected in HTTP request (content=inside.htm)
- **Note:** The presence of a user-controlled parameter does not confirm exploitation, but indicates a potential injection point.

- **HTTP Status:** 302 Found
- **Referer** : http://demo.testfire.net/index.jsp?content=personal_other.htm
- **Cookie**: JSESSIONID=078E83B4443B106F9B616CA72E82F390

Figure 1: HTTP request and response highlighting a user-controlled parameter (content), session cookie (JSESSIONID), and server redirection behavior, indicating potential injection entry points and session management mechanisms.

- **Impact:**
  Successful exploitation may lead to unauthorized access to sensitive data, including user credentials, financial records, or internal application data. In severe cases, attackers could modify or delete database contents, resulting in data integrity loss.

- **Recommendation:**
  Implement parameterized queries (prepared statements) and enforce strict server-side input validation. Additionally, apply least-privilege principles to database accounts.

Cross-Site Scripting (Potential)

- **Risk:** High
- **Description**:
  The application reflects user input in HTTP responses without adequate encoding or sanitization. This allows attackers to inject and execute malicious JavaScript in the context of a victim's browser session.

Cross Site Scripting (Reflected)

URL:	http://demo.testfire.net/sendFeedback
Risk:	High
Confidence:	Medium
Parameter:	name
Attack:	<script>alert(1);</scRipt>
Evidence:	<script>alert(1);</scRipt>
CWE ID:	79
WASC ID:	8
Source:	Active (40012 - Cross Site Scripting (Reflected))
Input Vector:	Form Query

Description:

Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.

- **Impact:**
  Attackers may hijack user sessions, steal cookies, redirect users to malicious websites, or perform actions on behalf of the victim without their consent.
- **Recommendation:**
  Apply proper input validation and output encoding. Use frameworks or libraries that automatically escape user-generated content. Implement Content Security Policy (CSP) as an additional defense layer.

Missing Content Security Policy (CSP)

- **Risk:** Medium
- **Description:**
  The application does not define a Content Security Policy header, which is a critical browser security mechanism used to control the sources of executable scripts and resources.

Content Security Policy (CSP) Header Not Set

URL:	http://demo.testfire.net
Risk:	Medium
Confidence:	High
Parameter:	
Attack:	
Evidence:	
CWE ID:	693
WASC ID:	15
Source:	Passive (10038 - Content Security Policy (CSP) Header Not Set)
Alert Reference:	10038-1
Input Vector:	

Description:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.

- **Impact:**
  Without CSP, the application is more vulnerable to client-side attacks such as XSS, as browsers cannot restrict the execution of untrusted scripts.
- **Recommendation:**
  Define a strict CSP header to allow only trusted sources for scripts, styles, and other resources.

Missing Anti-Clickjacking Protection

- **Risk:** Medium
- **Description:**
  The application does not implement anti-clickjacking protections such as the X-Frame-Options header or the frame-ancestors directive within a Content Security Policy (CSP). This allows the application to be embedded an iframe on external websites.

Missing Anti-clickjacking Header

URL:	http://demo.testfire.net
Risk:	Medium
Confidence:	Medium
Parameter:	x-frame-options
Attack:	
Evidence:	
CWE ID:	1021
WASC ID:	15
Source:	Passive (10020 - Anti-clickjacking Header)
Alert Reference:	10020-1
Input Vector:	

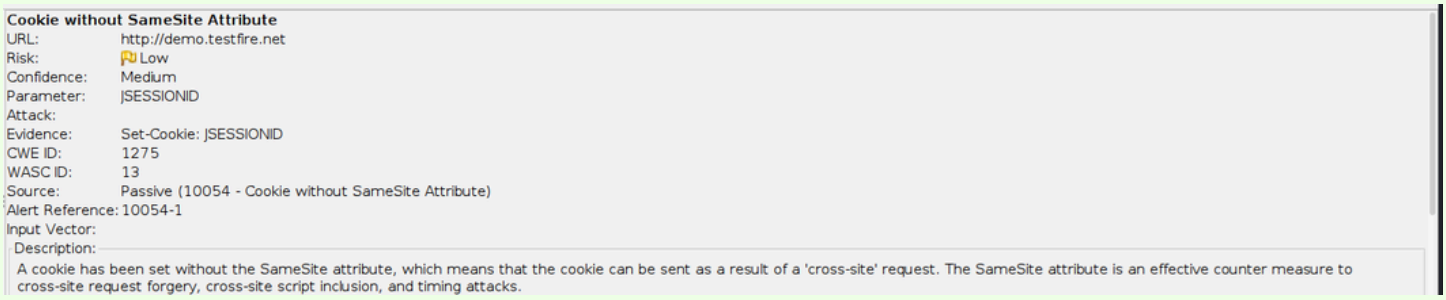
Description:

The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

- **Impact:**
  Attackers can trick users into interacting with hidden or disguised elements by embedding the application within a malicious page. This may result in unintended actions being performed without the user's knowledge, such as clicking buttons or submitting sensitive data.
- **Recommendation:**
  Implement the X-Frame-Options header with values such as DENY or SAMEORIGIN, or configure the frame-ancestors directive within a properly defined Content Security Policy (CSP) to restrict which domains are allowed to embed the application.

Cookie Without SameSite Attribute

- **Risk: Medium**
- **Description**:
  Session cookies are configured without the SameSite attribute, which defines whether cookies are included in cross-site requests initiated by third-party websites.



Cookie without SameSite Attribute

URL:	http://demo.testfire.net
Risk:	Low
Confidence:	Medium
Parameter:	JSESSIONID
Attack:	
Evidence:	Set-Cookie: JSESSIONID
CWE ID:	1275
WASC ID:	13
Source:	Passive (10054 - Cookie without SameSite Attribute)
Alert Reference:	10054-1
Input Vector:	

Description:

A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.

- **Impact:**
  This increases the risk of Cross-Site Request Forgery (CSRF) attacks, where an attacker can exploit an authenticated user's session to perform unauthorized actions on their behalf without their knowledge or consent.
- **Recommendation:**
  Set the SameSite attribute to "Strict" or "Lax" depending on application requirements. Additionally, ensure cookies are configured with the Secure and HttpOnly flags to enhance overall session security.

Information Disclosure (In Page Banner)

- **Risk:** Low
- **Description:** The application reveals internal behavior and structure through HTTP responses, particularly when handling invalid user-controlled input (e.g., content parameter), which may disclose information about server configuration and application routing.
- **Evidence:** User-controlled parameter (content=inside_about.htm) detected in HTTP referer, with server returning a 404 Not Found response, indicating improper handling of manipulated input and potential exposure of application structure.

Figure 6: HTTP request referencing a user-controlled parameter (content) resulting in a 404 response, demonstrating how manipulated input affects server behavior and may reveal application structure.

- **Impact:**
  Although not directly critical, this information can assist attackers in reconnaissance activities by enabling them to identify underlying technologies and tailor more targeted attacks (fingerprinting).
- **Note:** While this issue is classified as low risk, it can be leveraged in combination with other vulnerabilities to increase attack effectiveness.
- **Recommendation:**
  Remove or obfuscate unnecessary system information from HTTP responses, headers, and page content. Configure the server to

limit disclosure of version details and internal implementation data.

Missing Subresource Integrity (SRI)

- **Risk:** Low
- **Description:**
  The application loads external scripts or stylesheets without verifying their integrity using Subresource Integrity (SRI) attributes.

Sub Resource Integrity Attribute Missing

URL:	http://demo.testfire.net/index.jsp?content=personal_investments.htm
Risk:	Medium
Confidence:	High
Parameter:	
Attack:	
Evidence:	<script src="http://demo-analytics.testfire.net/urchin.js" type="text/javascript"></script>
CWE ID:	345
WASC ID:	15
Source:	Passive (90003 - Sub Resource Integrity Attribute Missing)
Input Vector:	

Description:

The integrity attribute is missing on a script or link tag served by an external server. The integrity tag prevents an attacker who have gained access to this server from injecting a malicious content.

- **Impact:**
  If a third-party resource is compromised, malicious code could be executed within the application without detection.
- **Recommendation:**
  Use integrity and crossorigin attributes when loading external resources to ensure they have not been tampered with.

▢ Security Misconfiguration (Missing Security Headers)

- **Risk:** Medium
- **Description:**
  The application does not implement essential HTTP security headers that help protect against common web-based attacks such as Cross-Site Scripting (XSS), clickjacking, and protocol downgrade attacks. These headers provide an additional layer of security by instructing the browser on how to handle content securely.
- **Evidence:**
- Analysis of HTTP response headers shows that the following security headers are missing:
- Content-Security-Policy (CSP)
- X-Frame-Options
- Strict-Transport-Security (HSTS)
- X-Content-Type-Options

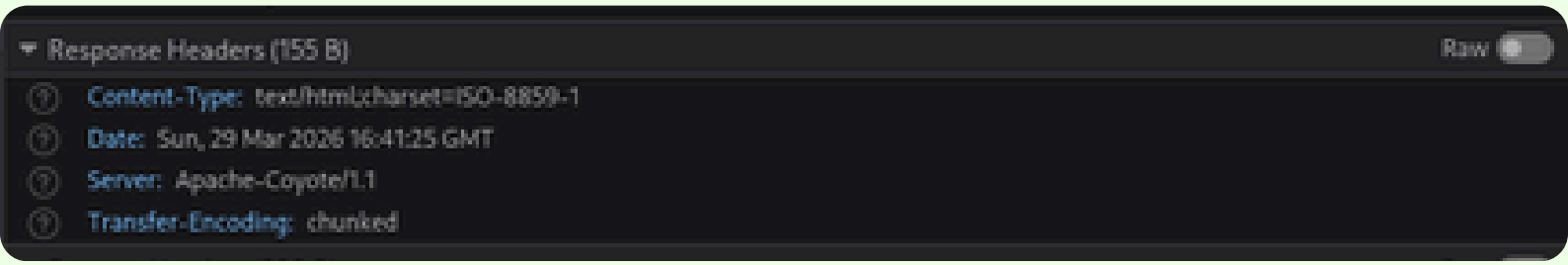


Figure 8: HTTP response headers showing absence of critical security headers, indicating security misconfiguration.

- **Impact:**
  The absence of these headers increases the attack surface of the application and makes it more vulnerable to client-side attacks such as XSS, clickjacking, and content injection.
- **Recommendation:**
- Implement the following security headers:
- **Content-Security-Policy (CSP):** Restrict trusted sources of scripts and resources

- **X-Frame-Options:** Prevent clickjacking attacks (use DENY or SAMEORIGIN)
- **Strict-Transport-Security (HSTS):** Enforce secure HTTPS connections
- **X-Content-Type-Options:** Prevent MIME-type sniffing (nosniff)

• Conclusion

The assessment identified multiple security weaknesses across the application, including high-risk issues such as SQL Injection and potential Cross-Site Scripting. While some findings are classified as medium and low risk, they collectively increase the attack surface and may be leveraged in combination. Immediate remediation is recommended to enhance the overall security posture of the application.

Thanks